

Global Warming Science

<https://courses.seas.harvard.edu/climate/eli/Courses/EPS101/>

Droughts!

Please use the template below to answer the workshop questions. “XX” indicates places where you need to complete/write code or add a discussion.

Your name:

```
[ ]: # Load libraries and data:
import numpy as np
import pickle
import matplotlib.pyplot as plt
from scipy import stats
from scipy.integrate import solve_ivp
from scipy import signal
from scipy import optimize
import warnings
# cortopy allows to plot data over maps in various spherical prjections"
import cartopy.crs as ccrs
from datetime import datetime

# load the data from a pickle file:
with open('./droughts_variables.pickle', 'rb') as file:
    while 1:
        try:
            d = pickle.load(file)
        except (EOFError):
            break
        # print information about each extracted variable:
        for key in list(d.keys()):
            if isinstance(d[key], list):
                print("extracting pickled variable: name=", key\
                    , "; len=", len(d[key]))
            else:
                print("extracting pickled variable: name=", key\
                    , "; size=", d[key].shape)
        globals().update(d)

fontsize=9
plt.rcParams['font.size'] = fontsize

print("done.")
```

Explanation of variables

PDSI_timeseries: a time series of the Palmer Drought Severity Index for the SW US

Tree-ring data:

(Note that all individual_tree_records_widths/ all individual_tree_records_widths_years are both lists. Each element of the list is an individual tree record/ time axis. The above “size” of these variables, is the

number of tree records in each list. The length of each record is different. Each record can be accessed and printed, as shown further below)

all_individual_tree_records_widths, all_individual_tree_records_years

[Tree ring data are in units of 0.01 mm]

analyzing drought projections of two climate models for the Sahel and SW US: model0 vs model1

(each variable contains a time axis and the corresponding quantity)

sahel_model0_evaporation, sahel_model0_moisture, sahel_model0_precipitation

sahel_model1_evaporation, sahel_model1_moisture, sahel_model1_precipitation

sw_us_model0_evaporation, sw_us_model0_moisture, sw_us_model0_precipitation,

sw_us_model1_evaporation, sw_us_model1_moisture, sw_us_model1_precipitation

1. Past and future of Sahel and Southwest United States droughts: Plot the provided results of the two climate models for 1850–2100, based on a projected RCP8.5 scenario, showing the following time series for the Sahel: (1) evaporation and precipitation (in m/yr), (2) evaporation minus precipitation, (3) soil moisture. Repeat, in three more panels, for the Southwest United States. Describe what you see in each panel, discussing the likelihood of future droughts in each region and the expected uncertainty.

```
[ ]: # plot:
fig1 = plt.figure(1,figsize=(15,10)); plt.clf()
plt.subplot(2,3,1)
plt.ylabel("m/yr")
plt.xlabel("Year")
plt.title("Sahel Evaporation and precipitation")
plt.plot(sahel_model0_evaporation[0,:], sahel_model0_evaporation[1,:], lw=2,
         label="model0 E")
plt.plot(sahel_model0_precipitation[0,:], sahel_model0_precipitation[1,:], lw=2,
         label="model0 P")
plt.plot(sahel_model1_evaporation[0,:], sahel_model1_evaporation[1,:], lw=2,
         label="model1 E")
plt.plot(sahel_model1_precipitation[0,:], sahel_model1_precipitation[1,:], lw=2,
         label="model1 P")
plt.legend()

plt.subplot(2,3,2)
plt.ylabel("m/yr")
plt.xlabel("Year")
plt.title("Sahel Evaporation minus Precipitation")
plt.plot(XX,XX)
plt.legend()

plt.subplot(2,3,3)
plt.ylabel("kg/m$^2$")
```

```

plt.xlabel("Year")
plt.title("Sahel soil moisture anomalies for both models")
XX # calculate and remove time-mean from each model [use np.mean()]
plt.plot(XX,XX)
plt.legend()

plt.subplot(2,3,4)
plt.ylabel("m/yr")
plt.xlabel("Year")
plt.title("SW US Evaporation and precipitation for both models")
plt.plot(XX,XX)
plt.legend()

plt.subplot(2,3,5)
plt.ylabel("m/yr")
plt.xlabel("Year")
plt.title("SW-US Evaporation minus Precipitation")
plt.plot(XX,XX)
plt.legend()

plt.subplot(2,3,6)
plt.ylabel("kg/m$^2$")
plt.xlabel("Year")
plt.title("SW-US soil moisture anomalies")
XX remove mean
plt.plot(XX,XX)
plt.legend()
plt.tight_layout()
plt.show()

```

2. Identifying ACC in a long-term Southwest United States drought record:

Explanation of the input tree ring data:

Note that all `individual_tree_records_widths`/ `all_individual_tree_records_years` are both lists. Each element of the list is an individual tree record/ time axis. The above “size” of these variables, is the number of tree records in each list. The length of each record is different. Each record can be accessed and printed, for example using:

```

[ ]: # print for example record #3:
print(" data=",all_individual_tree_records_widths[3])
print(" years=",all_individual_tree_records_years[3])

print("\n some of the data=",all_individual_tree_records_widths[3][3:8])
print(" and the corresponding years=",all_individual_tree_records_years[3][3:8])

```

(a) Plot the 2000 yr time series of PDSI from the North American Drought Atlas.

```

[ ]: pdsi=PDSI_timeseries[1,:]
years=PDSI_timeseries[0,:]

```

```

pdsi_threshold=-2 # a drought is defined as being less than this value

# plot the pdsi time series:
# -----
plt.figure(figsize=(15,5))
# plot the PDSI time series
plt.plot(XX, label="pdsi")
# add a thin dash line at pdsi=0:
plt.plot(XX, XX)
# add a thin dash line at pdsi=threshold value used to define droughts:
plt.plot(XX, XX)
plt.legend()
plt.xlabel("XX")
plt.ylabel("XX")
plt.title("PDSI averaged over South West US")
plt.tight_layout()

```

(b) Use non-parametric statistics to figure out if the number of droughts in the most recent $N=30$ yr in the record is unusual within 95%, defining a drought as being $PDSI < -2$, and using 10,000 realizations.

```

[ ]: # calculate number of drought years in the last N_years of data:
N_years=30
# initialize counter:
counter=pdsi*0.0
XX # set elements of counter that correspond to pdsi<=pdsi_threshold to 1
XX print counter to understand what it represents, and then
# calculate and print the total number of drought events in the last N_years,
# using the above variable counter
XX

# scramble years in the data to see how many droughts randomly
# occur in the last 30 years and compare to the actual data
N_permutations=10000
num_droughts_in_last_N_years=np.zeros(N_permutations)
for i in range(0,N_permutations):
    counter=pdsi*0.0
    pdsi_randomized=np.random.permutation(pdsi)
    # count number of droughts in the last N_years:
    XX # set elements of counter that correspond to pdsi<=pdsi_threshold to 1
    num_droughts_in_last_N_years[i]=XX

# plot probability distribution function of number of drought events in last N years:
plt.figure(figsize=(6,6))
plt.hist(num_droughts_in_last_N_years, bins=list(range(0,20)));
plt.xlabel("XX")
plt.ylabel("XX")
plt.title("XX");

```

3. Tree-ring data: Plot (in thin gray lines) the time series of tree-ring width records for bigcone Douglas-fir trees from the San Bernardino Mountains as a function of time. Plot (in a thick, color line) the decadal bin-average. Comment on the scatter around the average and discuss the implications for the uncertainty of the reconstructed drought record based on these data.

Tree-ring data:

(Note that all `individual_tree_records_widths`/`all_individual_tree_records_years` are both lists. Each element of the list is an individual tree record/ time axis. The “size” of these variables, is the number of tree records in each list. The length of each record is different. Each record can be accessed and printed, as shown further below)

```
[ ]: # plot the individual tree-ring records in gray:
plt.figure(figsize=(15,5))
# supplement the following line with a for loop to plot all records:
for XX:
    XX
plt.plot(all_individual_tree_records_years[XX][:],
         ,all_individual_tree_records_widths[XX][:]\
         ,linewidth=0.5,color="grey")
plt.xlabel("year")
plt.ylabel("width")
plt.title("tree-ring width record")

# calculate the bin-averaged record by average over decadal-bins
# first combine all tree record into a single one:
years = []; widths=[];
for sublist in all_individual_tree_records_years:
    for item in sublist:
        years.append(item)
for sublist in all_individual_tree_records_widths:
    for item in sublist:
        widths.append(item)

# now calculate bin-average:
bin_edges_input=np.arange(1550,2010,10)
bin_means, bin_edges, binnumber \
    = stats.binned_statistic(years,widths, statistic='mean',bins=bin_edges_input)
years_axis=(bin_edges[0:-1]+bin_edges[1:])*0.5

# now plot over the individual tree records:
plt.plot(XX,XX,'r-',linewidth=2,label="bin-average");
```

Discussion: XX

(b) Create a normally-distributed random data set for a variable x with a zero mean and std of 1, and with the number of points equal to $365 \times 20 \times 20 \times 10^i$, where $i = 0, 1, 2, 3$. Plot the probability distribution function histogram $PDF(x)$ for $-4 < x < 4$ in each case on a log y-axis. Discuss how the estimate of the tail of the distribution improves with an increasing number of data points.

```
[ ]: plt.figure(dpi=300)
for i in range(0,4):
    N=365*20*10**i
```

```

data=np.random.normal(loc=0.0, scale=1.0, size=N)
plt.subplot(4,1,i+1)
hist, bin_edges = np.histogram(data, bins=150, density=True, range=[-4,4])
bin_centers = XX
width=XX
h=plt.bar(XX, XX, width=width)
plt.yscale('log')
plt.title("N=%g, or %g years" % (N,20*10**i) )

plt.tight_layout()

```

Discussion: XX

4. Bucket model for soil moisture:

(a) Run the bucket model with a precipitation rate of $P_1 = 2.2$ m/yr and then $P_2 = 1.8$ m/yr. Plot $W(t)$ and $P-E$ as a function of time. Why doesn't the soil completely dry when the precipitation rate is decreased from the value P_1 that led to the first equilibrium to the lower value P_2 ? Describe and explain the stages seen in the solution time series.

```

[ ]: #####
# functions
#####

# saturation specific humidity:
def q_sat(T,P):
    # saturation specific humidity (gr water vapor per gram moist air):
    # inputs:
    # T: temperature, in K
    # P: pressure, in mb

    R_v = 461 # Gas constant for moist air = 461 J/(kg*K)
    R_d = 287 # Gas constant 287 J K^-1 kg^-1
    TT = T-273.15 # Kelvin to Celsius
    # Saturation water vapor pressure (mb) from Emanuel 4.4.14 p 116-117:
    ew = 6.112*np.exp((17.67 * TT) / (TT + 243.5))
    # saturation mixing ratio (gr water vapor per gram dry air):
    rw = (R_d / R_v) * ew / (P - ew)
    # saturation specific humidity (gr water vapor per gram moist air):
    qw = rw / (1 + rw)
    return qw

def Precip(t):
    # precipitation rate
    return Precip0

# Right hand side of bucket model ODE to be integrated:
def right_hand_side(t, W):
    # t: time
    # W: soil moisture
    EO=rho_air*C_D*V*q_sat(T,1000)*RH/rho_water
    if W >= W_FC and Precip(t)-EO > 0.0:
        dW_dt=np.nan #XX
    elif W >= W_k:

```

```

        dW_dt=np.nan #XX
    else:
        dW_dt=np.nan #XX
    return dW_dt

# ODE solver:
def solve_ODE():
    """
    Solve the ODE for soil moisture econtent
    """
    tspan=(times_save[0],times_save[-1])
    sol = solve_ivp(fun=lambda t,W: right_hand_side(t,W) \
        ,vectorized=False,y0=W0,t_span=tspan,t_eval=times_save,rtol=1.e-6)
    time_array=sol.t[:]
    W_array=sol.y[0,:]

    # calculate the RHS of the differencial equation for all times, for plotting:
    dW_dt_array=0.0*W_array
    for i in range(0,len(W_array)):
        t=1.0*time_array[i]
        W=1.0*W_array[i]
        dW_dt_array[i]=right_hand_side(t, W)

    return W_array, time_array/YEAR, dW_dt_array

#####
# set parameters
#####

YEAR = 31536000 # a year in seconds
DAY=86400
YEARS_TO_RUN = 4
# time step specifying resolutio for plotting:
dt=YEARS_TO_RUN*YEAR/1000
times_save=np.arange(0,YEARS_TO_RUN*YEAR,dt)
C_D=0.5e-3
RH=0.6
rho_air=1.225 # kg/m^3
rho_water=1000 # kg/m^3
V=10 # wind velocity m/s
W_FC=0.15
W_k=0.75*W_FC
T=25+273.15 # temperature in Kelvin

# initial values for soil moisture, must be smaller than W_FC:
W0 = np.array([0.14]) # m, solve_ivp really wants this to be a numpy array

#####
# Main program
#####

```

```

# solve ODE:
Precip1=1.8/YEAR # in units of m/s
Precip0=Precip1
W_array1, time_array1, dW_dt_array1 = solve_ODE()
Precip2=2.2/YEAR # in units of m/s
Precip0=Precip2
W_array2, time_array2, dW_dt_array2 = solve_ODE()

# plot soil water content:
fig=plt.figure(1,figsize=(6,2),dpi=300);
plt.clf()
plt.subplot(1,2,1)
plt.plot(time_array1, W_array1, "-", color='r', label="soil moisture, □
↳P="+repr(Precip1*YEAR)+" m/year")
plt.plot(time_array2, W_array2, "-", color='b', label="soil moisture, □
↳P="+repr(Precip2*YEAR)+" m/year")
plt.plot(time_array1, 0*W_array1, "--", color='black', lw=0.5)
plt.title('soil water content (m)')
plt.ylabel('W (m)')
plt.xlabel('Time (years)')
plt.legend(loc='upper right')

# plot P-E, rhs of soil moisture equation:
plt.subplot(1,2,2)
plt.plot(time_array1, dW_dt_array1*YEAR, "-", color='r', label="dW/dt, □
↳P="+repr(Precip1*YEAR)+" m/year")
plt.plot(time_array2, dW_dt_array2*YEAR, "-", color='b', label="dW/dt, □
↳P="+repr(Precip2*YEAR)+" m/year")
plt.plot(time_array1, 0*W_array1, "--", color='black', lw=0.5)
plt.title('P-E (m/year)')
plt.ylabel('dW/dt (m/year)')
plt.xlabel('Time (years)')
plt.legend(loc='lower right')
plt.tight_layout()
plt.show()

```

(b) Optional extra credit: Drive the bucket model with a precipitation rate of 2 m/yr plus a white noise time series with a zero mean and a specified std. Compare the occurrence of meteorological versus hydrological drought events from the model output via their PDFs and power spectra, and rationalize your results Hint: a red noise sequence f_n is produced from a gaussian random white noise ν_n as follows,

$$R = \exp(-dt/T_{correlation})$$

$$f_n = R \cdot f_{n-1} + \sqrt{1 - R^2} \cdot \nu_{n-1}$$

where $T_{correlation}$ is the desired correlation time of the noise, and dt is the time step between two elements in the desired red noise sequence.

```

[ ]: # bucket model driven with stochastic precipitation:

YEAR = 31536000 # a year in seconds
DAY=86400
YEARS_TO_RUN = 100
MONTH=XX
# time step specifying resolution for plotting:
dt=YEARS_TO_RUN*YEAR/20000
times_save=np.arange(0,YEARS_TO_RUN*YEAR,dt)
N=len(times_save)

# creat a random sequence  $\nu_n$ :
np.random.seed(0)
random=np.random.randn(N)

# creaete a red-noise precipitation record:
rednoise=np.zeros(N)
T_corr=MONTH
R=XX
for n in range(1,N):
    rednoise[n]=XX
# dimensional precipitation red noise time series:
precipitation_fixed_points=Precip0*(1+0.05*rednoise)

# redefine precip function to include a stochastic componenet
def Precip(t):
    # returns precipitation rate at time t.

    # interpolate from red noise sequence to time t:
    precipitation=np.interp(t,times_save,precipitation_fixed_points)

    return precipitation

tspan=(times_save[0],times_save[-1])
# initial values for soil moisture, must be smaller than  $W_{FC}$ :
W0 = np.array([0.11]) # m, solve_ivp really wants this to be a numpy array
sol = solve_ivp(fun=lambda t,W: right_hand_side(t,W) \
    ,vectorized=False,y0=W0,t_span=tspan,t_eval=times_save,rtol=1.e-6)
time_array=sol.t[:]
W_array=sol.y[0,:]

fig=plt.figure(figsize=(6,4),dpi=300)
plt.subplot(3,2,(1,2))
# precipitation forcing vs time:
plt.plot(XX)
plt.xlabel("Time (years)")
plt.ylabel("Precip (m/yr)");
plt.xlim(80,90)

plt.subplot(3,2,(3,4))

```

```

# soil moisture vs time:
plt.plot(XX)
plt.xlabel("Time (years)")
plt.ylabel("W (m)");
plt.xlim(80,90)

plt.subplot(3,2,5)
f,psd_random=signal.welch(random,dt)
plt.semilogy(psd_random[1:])
plt.title("spectrum for precipitation")
plt.xlabel("frequency")

plt.subplot(3,2,6)
XX
plt.title("spectrum for soil moisture")
plt.xlabel("frequency")

plt.tight_layout()

```

5. Optional extra credit: Composite analyses of drought years:

(a) Analyze the provided results from a long control run of a climate model to calculate and contour the rainy season averages (composites) of sea level pressure and precipitation anomalies for times in which the area-mean precipitation over the region of your city of birth is 1 std below its long-term average. Describe the pattern you calculate and discuss the expected impacts on agriculture and water availability for that region.

(b) Plot, describe, and explain the precipitation over the tropical Pacific, from Australia to South America, averaged over times corresponding to El Niño versus La Niña events. Define the times of these events as the NINO3.4 index being above and below 1 std.

```

[ ]: # reproduce something like textbook Figure 12.2
      # you need to download the data yourself from the CMIP6 archive!

XX

```